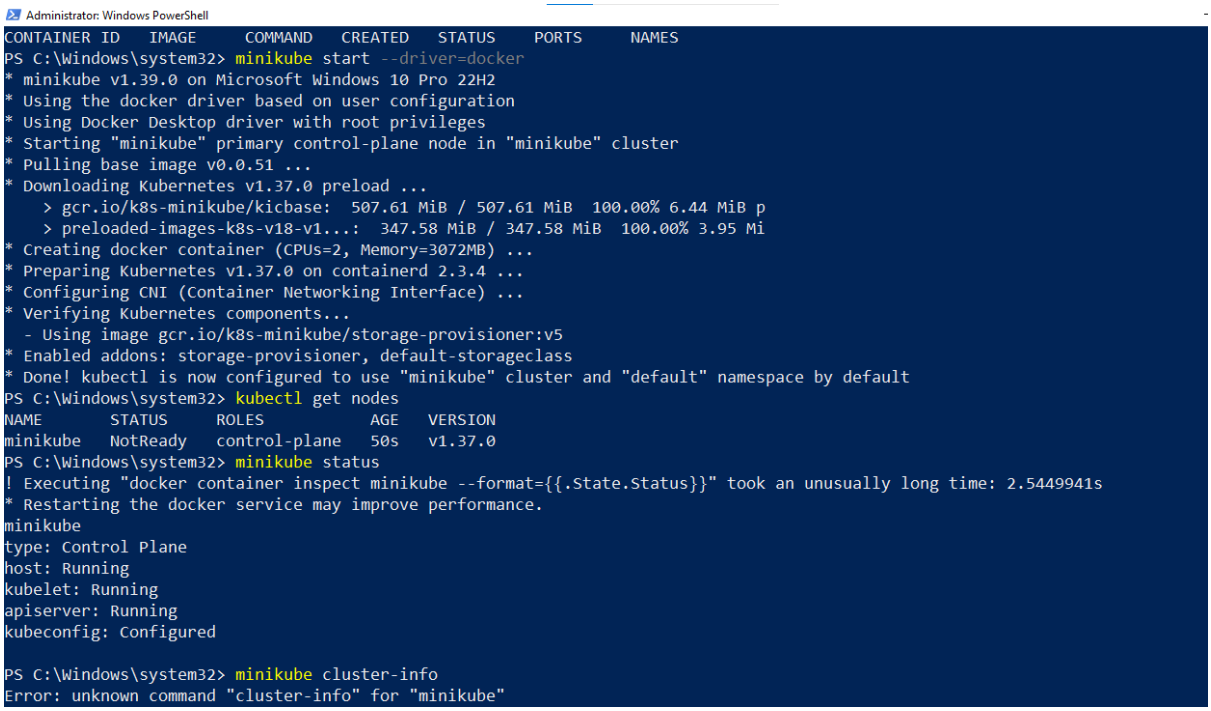
Successfully Setup Minikube in local machine.

- **Installed Minikube on Windows using Winget.**
- **Verified Minikube version (v1.39.0).**
- **Verified Docker and Kubernetes connectivity.**
- **Checked Kubernetes control-plane status.**
- **Verified CoreDNS service.**
- **Checked Kubernetes cluster information using kubectl**.

Minikube version.



Kubernetes cluster-info.

```
Kubernetes control plane is running at https://127.0.0.1:53812
CoreDNS is running at https://127.0.0.1:53812/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
PS C:\Windows\system32> kubectl get nodes -o wide
NAME       STATUS   ROLES           AGE     VERSION   INTERNAL-IP    EXTERNAL-IP   OS-IMAGE                         KERNEL-VERSION
                    CONTAINER-RUNTIME
minikube   Ready    control-plane   5m21s   v1.37.0   192.168.49.2   <none>        Debian GNU/Linux 12 (bookworm)   6.18.33.2-microso
t-standard-WSL2 (amd64)   containerd://2.3.4
PS C:\Windows\system32> kubectl get pods -A
NAMESPACE     NAME                               READY   STATUS    RESTARTS   AGE
kube-system   coredns-559f6c778d-z5g2k           1/1     Running   0          5m26s
kube-system   etcd-minikube                      1/1     Running   0          5m40s
kube-system   kindnet-scql7                      1/1     Running   0          5m25s
kube-system   kube-apiserver-minikube            1/1     Running   0          5m40s
kube-system   kube-controller-manager-minikube   1/1     Running   0          5m40s
kube-system   kube-proxy-gcfnp                   1/1     Running   0          5m26s
kube-system   kube-scheduler-minikube            1/1     Running   0          5m40s
kube-system   storage-provisioner                1/1     Running   0          5m17s
PS C:\Windows\system32> kubectl get nodes output.
Error from server (NotFound): nodes "output." not found
PS C:\Windows\system32> kubectl get nodes output
Error from server (NotFound): nodes "output" not found
PS C:\Windows\system32> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED         STATUS         PORTS
                                                                                                           NAMES
c54a12d89353   gcr.io/k8s-minikube/kicbase:v0.0.51   "/usr/local/bin/entr…"   9 minutes ago   Up 9 minutes   127.0.0.1:53813->22/tcp,
127.0.0.1:53814->2376/tcp, 127.0.0.1:53816->5000/tcp, 127.0.0.1:53812->8443/tcp, 127.0.0.1:53815->32443/tcp   minikube
PS C:\Windows\system32>
```

Docker version

```
127.0.0.1:53814->2376/tcp, 127.0.0.1:53816->5000/tcp, 127.0.0.1:53812->8443/tcp, 127.0.0.1:53815->32443/tcp   minikube
PS C:\Windows\system32> minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

Kubernetes Node Configuration (Master and Worker Nodes)

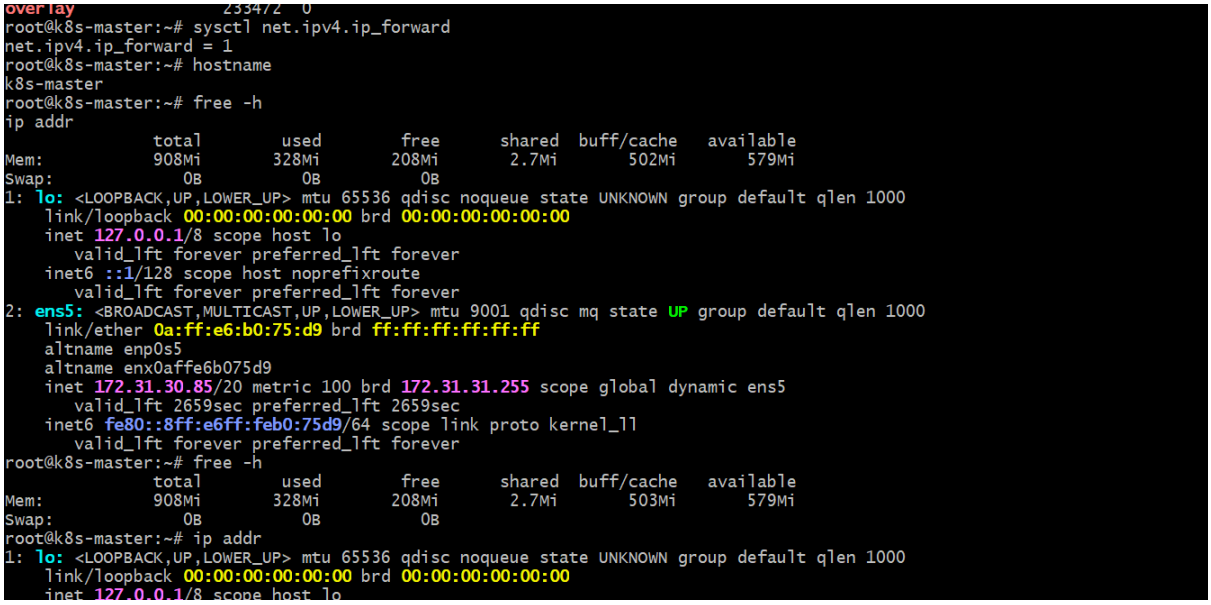
Configuration completed

Step-by-step points

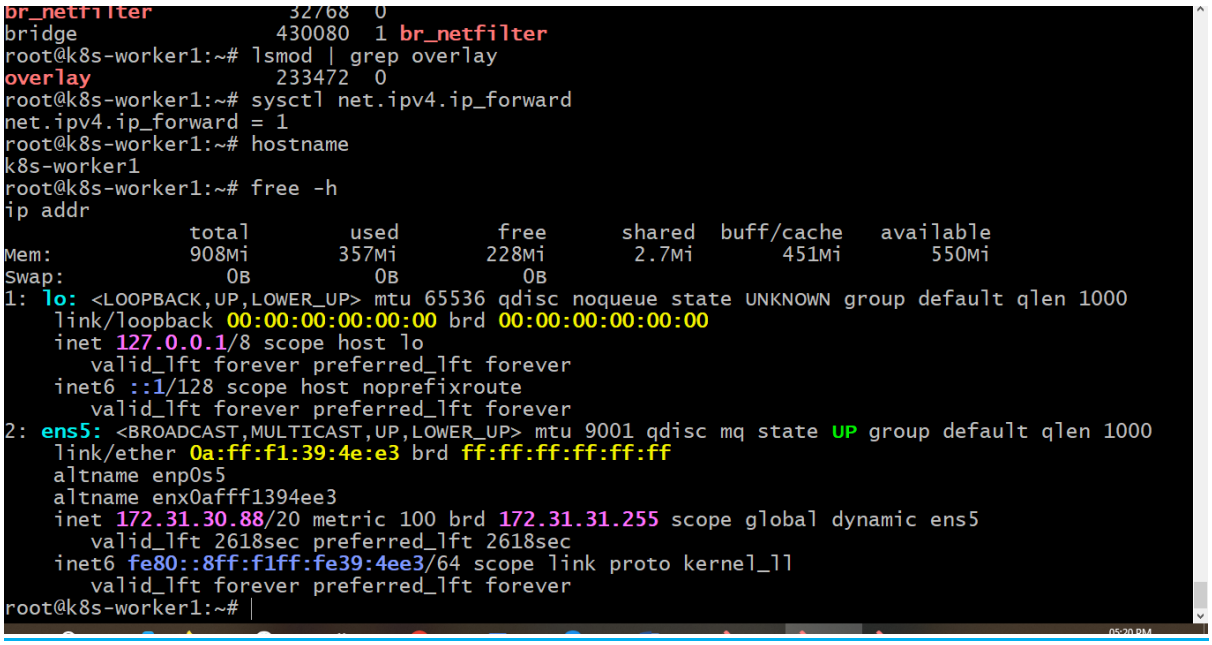
1. Connected to all three Ubuntu EC2 instances using SSH.
2. Updated Ubuntu packages using `apt update` and `apt upgrade`.
3. Disabled swap memory on all three nodes.
4. Configured persistent swap disabling through `/etc/fstab`.
5. Loaded the `overlay` kernel module.
6. Loaded the `br_netfilter` kernel module.
7. Created `/etc/modules-load.d/k8s.conf` to load required kernel modules automatically.
8. Created `/etc/sysctl.d/k8s.conf` for Kubernetes network configuration.
9. Enabled IPv4 packet forwarding.

10. Enabled bridge network traffic processing through iptables.
11. Applied kernel network settings using `sysctl --system`.
12. Verified kernel modules and IP forwarding on all three nodes.

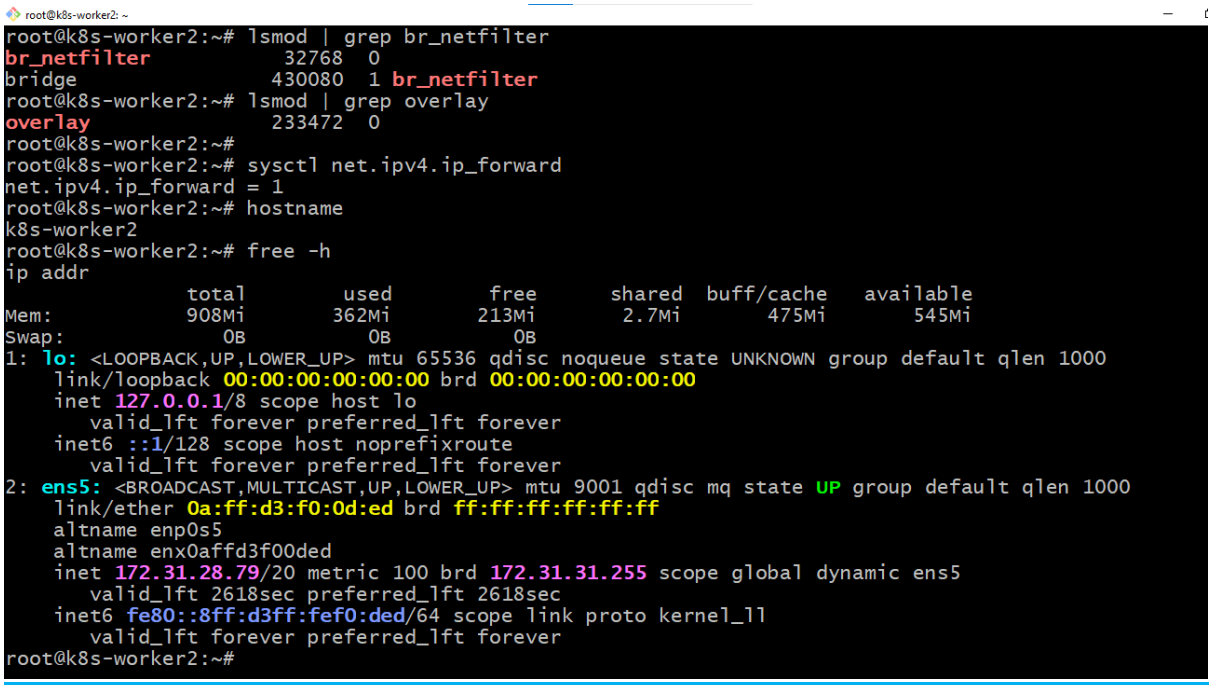
K8s-Master configured



K8s-worker1



K8s-worker2



Installation and Configuration of Containerd on Kubernetes Master and Worker Nodes

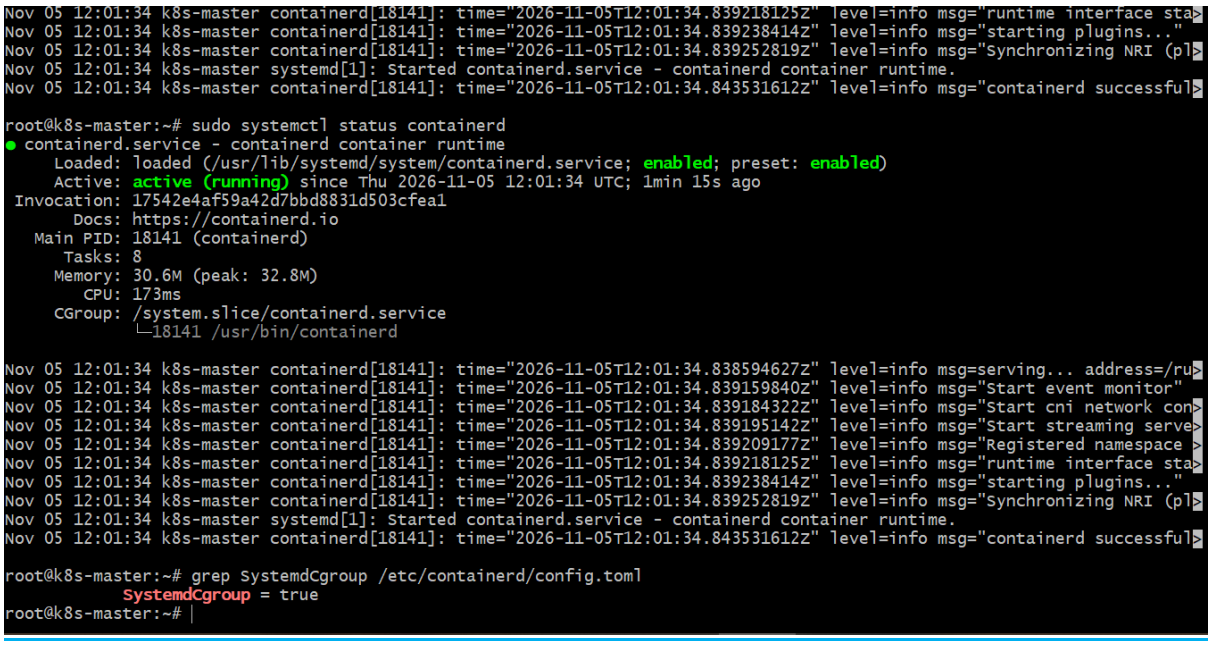
- **Installed containerd on master and worker nodes.**
- **Generated the default containerd configuration.**
- **Enabled SystemdCgroup configuration.**
- **Restarted containerd service.**
- **Enabled containerd at system startup.**
- **Verified containerd service status.**

K8s-master

K8s-master

`SystemdCgroup = true`.

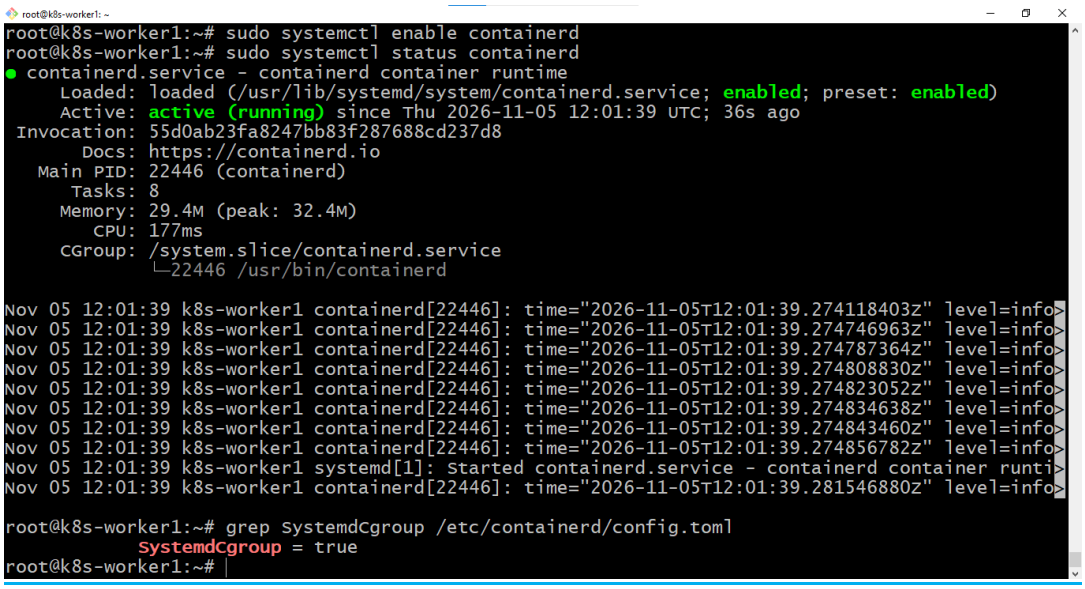
Containerd showing `active (running)`



K8s-worker1

`SystemdCgroup = true`.

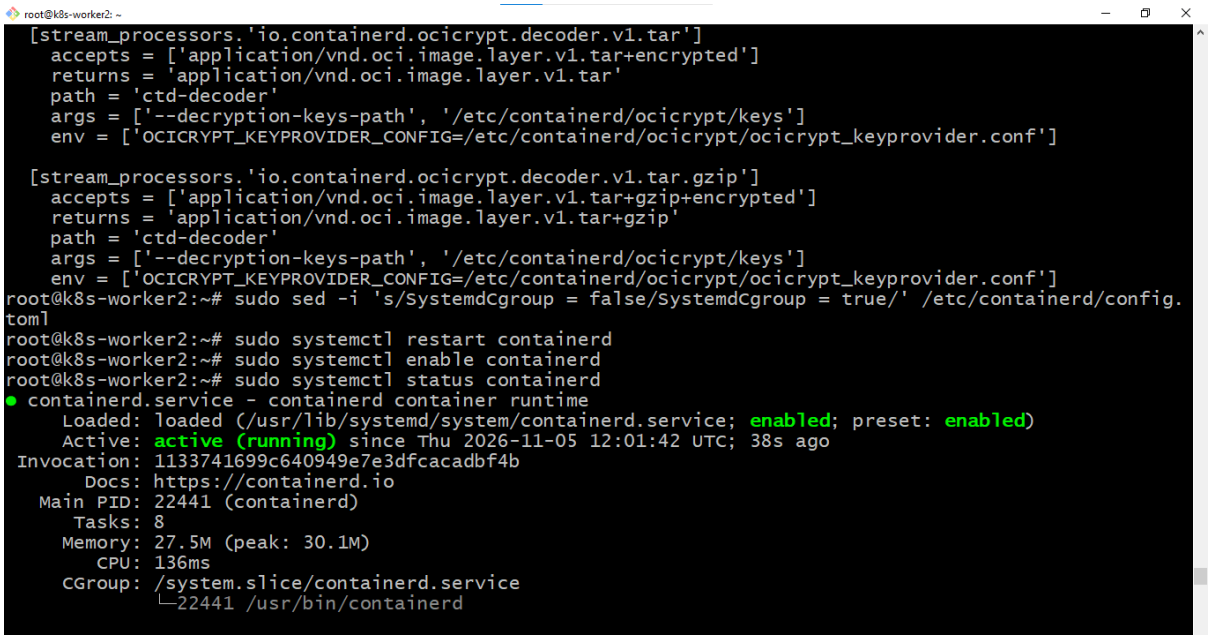
Containerd showing `active (running)`

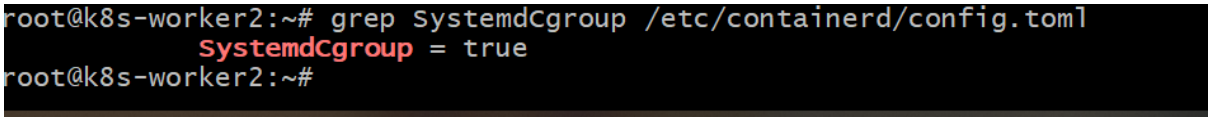


K8s-worker2

`SystemdCgroup = true.`

Containerd showing `active (running)`





Installation of Kubernetes Components on Master and Worker Nodes

1. Added the official Kubernetes package repository.
2. Configured the Kubernetes repository signing key.
3. Installed kubeadm, kubelet and kubectl.
4. Held Kubernetes package versions to prevent unintended upgrades.
5. Enabled the kubelet service.
6. Verified Kubernetes component versions on all three nodes.

Install required dependencies

sudo apt-get install -y apt-transport-https ca-certificates curl gpg

Create keyring directory

sudo mkdir -p -m 755 /etc/apt/keyrings

Download Kubernetes repository key

curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.34/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

Add Kubernetes repository

echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.34/deb/ /' | sudo tee /etc/apt/sources.list.d/kubernetes.list

Install Kubernetes components

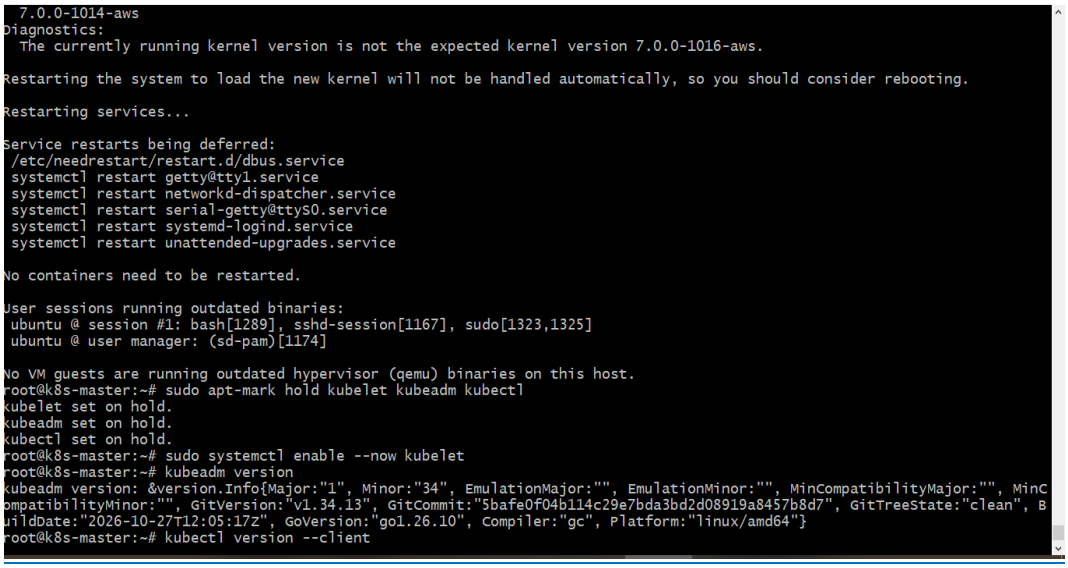
sudo apt-get install -y kubelet kubeadm kubectl

Enable kubelet

sudo systemctl enable --now kubelet

sudo systemctl status kubelet

k8s-master



K8s-worker1

```
 /etc/needrestart/restart.d/dbus.service
 systemctl restart getty@tty1.service
 systemctl restart networkd-dispatcher.service
 systemctl restart serial-getty@ttyS0.service
 systemctl restart systemd-logind.service
 systemctl restart unattended-upgrades.service

No containers need to be restarted.

User sessions running outdated binaries:
 ubuntu @ session #1: bash[1286], sshd-session[1164], sudo[1320,1322]
 ubuntu @ user manager: (sd-pam)[1171]

No VM guests are running outdated hypervisor (qemu) binaries on this host.
root@k8s-worker1:~# sudo apt-mark hold kubelet kubeadm kubectl
kubelet set on hold.
kubeadm set on hold.
kubectl set on hold.
root@k8s-worker1:~# sudo systemctl enable --now kubelet
root@k8s-worker1:~# kubeadm version
kubeadm version: &version.Info{Major:"1", Minor:"34", EmulationMajor:"", EmulationMinor:"", MinC
ompatibilityMajor:"", MinCompatibilityMinor:"", GitVersion:"v1.34.13", GitCommit:"5bafe0f04b114c
29e7bda3bd2d08919a8457b8d7", GitTreeState:"clean", BuildDate:"2026-10-27T12:05:17Z", GoVersion:"
go1.26.10", Compiler:"gc", Platform:"linux/amd64"}
root@k8s-worker1:~# kubectl version --client
Client Version: v1.34.13
Kustomize Version: v5.7.1
root@k8s-worker1:~#
```

K8s-worker2

```
Service restarts being deferred:
 /etc/needrestart/restart.d/dbus.service
 systemctl restart getty@tty1.service
 systemctl restart networkd-dispatcher.service
 systemctl restart serial-getty@ttyS0.service
 systemctl restart systemd-logind.service
 systemctl restart unattended-upgrades.service

No containers need to be restarted.

User sessions running outdated binaries:
 ubuntu @ session #1: bash[1286], sshd-session[1164], sudo[1320,1322]
 ubuntu @ user manager: (sd-pam)[1171]

No VM guests are running outdated hypervisor (qemu) binaries on this host.
root@k8s-worker2:~# sudo apt-mark hold kubelet kubeadm kubectl
kubelet set on hold.
kubeadm set on hold.
kubectl set on hold.
root@k8s-worker2:~# sudo systemctl enable --now kubelet
root@k8s-worker2:~# kubeadm version
kubeadm version: &version.Info{Major:"1", Minor:"34", EmulationMajor:"", EmulationMinor:"", MinCompatib
ilityMajor:"", MinCompatibilityMinor:"", GitVersion:"v1.34.13", GitCommit:"5bafe0f04b114c29e7bda3bd2d08
919a8457b8d7", GitTreeState:"clean", BuildDate:"2026-10-27T12:05:17Z", GoVersion:"go1.26.10", Compiler:
"gc", Platform:"linux/amd64"}
root@k8s-worker2:~# kubectl version --client
Client Version: v1.34.13
Kustomize Version: v5.7.1
root@k8s-worker2:~#
```

Initialization of Kubernetes Master and Installation of Flannel CNI

1. Initialized Kubernetes control plane using kubeadm.
2. Configured kubectl using admin.conf.
3. Installed Flannel CNI for pod networking.
4. Verified Kubernetes system pods and master node status.

```
root@k8s-master:~# sudo kubeadm init --pod-network-cidr=10.244.0.0/16
I1106 10:11:26.586338    1374 version.go:260] remote version is much newer: v1.37.2; falling back to: stable-1.34
[init] Using Kubernetes version: v1.34.13
[preflight] Running pre-flight checks
[preflight] Pulling images required for setting up a Kubernetes cluster
[preflight] This might take a minute or two, depending on the speed of your internet connection
[preflight] You can also perform this action beforehand using 'kubeadm config images pull'
[certs] Using certificateDir folder "/etc/kubernetes/pki"
[certs] Generating "ca" certificate and key
[certs] Generating "apiserver" certificate and key
[certs] apiserver serving cert is signed for DNS names [k8s-master kubernetes kubernetes.default kubernetes.default.svc kubernetes.def
ault.svc.cluster.local] and IPs [10.96.0.1 172.31.30.85]
[certs] Generating "apiserver-kubelet-client" certificate and key
[certs] Generating "front-proxy-ca" certificate and key
[certs] Generating "front-proxy-client" certificate and key
[certs] Generating "etcd/ca" certificate and key
[certs] Generating "etcd/server" certificate and key
[certs] etcd/server serving cert is signed for DNS names [k8s-master localhost] and IPs [172.31.30.85 127.0.0.1 ::1]
[certs] Generating "etcd/peer" certificate and key
[certs] etcd/peer serving cert is signed for DNS names [k8s-master localhost] and IPs [172.31.30.85 127.0.0.1 ::1]
[certs] Generating "etcd/healthcheck-client" certificate and key
[certs] Generating "apiserver-etcd-client" certificate and key
[certs] Generating "sa" key and public key
[kubeconfig] Using kubeconfig folder "/etc/kubernetes"
[kubeconfig] Writing "admin.conf" kubeconfig file
[kubeconfig] Writing "super-admin.conf" kubeconfig file
[kubeconfig] Writing "kubelet.conf" kubeconfig file
[kubeconfig] Writing "controller-manager.conf" kubeconfig file
[kubeconfig] Writing "scheduler.conf" kubeconfig file
[etcd] Creating static Pod manifest for local etcd in "/etc/kubernetes/manifests"
[control-plane] Using manifest folder "/etc/kubernetes/manifests"
[control-plane] Creating static Pod manifest for "kube-apiserver"
[control-plane] Creating static Pod manifest for "kube-controller-manager"
[control-plane] Creating static Pod manifest for "kube-scheduler"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
```

`kubectl get nodes` showing Ready.

```
Then you can join any number of worker nodes by running the following on each as root:

kubeadm join 172.31.30.85:6443 --token u9m3ka.7enokwokx7m5wk1m \
        --discovery-token-ca-cert-hash sha256:313004e1b8b297c63435816f7152dcb6ca23f94d946de96af23629624bc4438d
root@k8s-master:~# mkdir -p $HOME/.kube

sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config

sudo chown $(id -u):$(id -g) $HOME/.kube/config
root@k8s-master:~# kubectl get nodes
NAME         STATUS     ROLES           AGE   VERSION
k8s-master   NotReady   control-plane   41s   v1.34.13
root@k8s-master:~# ^C
root@k8s-master:~# kubectl apply -f https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml
namespace/kube-flannel created
serviceaccount/flannel created
clusterrole.rbac.authorization.k8s.io/flannel created
clusterrolebinding.rbac.authorization.k8s.io/flannel created
configmap/kube-flannel-cfg created
daemonset.apps/kube-flannel-ds created
root@k8s-master:~# kubectl get pods -A
NAMESPACE      NAME                                 READY   STATUS    RESTARTS   AGE
kube-flannel   kube-flannel-ds-sz8jg                1/1     Running   0          13s
kube-system    coredns-66bc5c9577-s2bf8             0/1     Pending   0          2m15s
kube-system    coredns-66bc5c9577-tvq8j             0/1     Pending   0          2m15s
kube-system    etcd-k8s-master                      1/1     Running   0          2m21s
kube-system    kube-apiserver-k8s-master            1/1     Running   0          2m22s
kube-system    kube-controller-manager-k8s-master   1/1     Running   0          2m21s
kube-system    kube-proxy-l7876                     1/1     Running   0          2m15s
kube-system    kube-scheduler-k8s-master            1/1     Running   0          2m21s
root@k8s-master:~# kubectl get nodes
NAME         STATUS   ROLES           AGE     VERSION
k8s-master   Ready    control-plane   2m46s   v1.34.13
```

joining Worker Nodes to Kubernetes Cluster

1. Connected the first worker node using the kubeadm join command.
2. Successfully started the kubelet service.
3. Completed TLS bootstrap for the worker node.
4. Registered the worker node with the Kubernetes control plane.

```
^C
root@k8s-master:~# kubectl get nodes -o wide
NAME          STATUS   ROLES           AGE   VERSION   INTERNAL-IP    EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION   CONTAINER-R
UNTIME
k8s-master    Ready    control-plane   21m   v1.34.13  172.31.30.85   <none>        Ubuntu 26.04.1 LTS   7.0.0-1016-aws   containerd:
//2.2.2
k8s-worker1   Ready    <none>          89s   v1.34.13  172.31.30.88   <none>        Ubuntu 26.04.1 LTS   7.0.0-1016-aws   containerd:
//2.2.2
k8s-worker2   Ready    <none>          96s   v1.34.13  172.31.28.79   <none>        Ubuntu 26.04.1 LTS   7.0.0-1016-aws   containerd:
//2.2.2
root@k8s-master:~#
```

Successful join output from worker1 and worker2

```
root@k8s-worker1:~# kubeadm join 172.31.30.85:6443 --token u9m3ka.7enokwokx7m5wk1m \
--discovery-token-ca-cert-hash sha256:313004e1b8b297c63435816f7152dcb6ca23f94d946de96af23629624bc4438d
[preflight] Running pre-flight checks
error: error execution phase preflight: couldn't validate the identity of the API Server: failed to request the
cluster-info ConfigMap: Get "https://172.31.30.85:6443/api/v1/namespaces/kube-public/configmaps/cluster-info?tim
eout=10s": context deadline exceeded
To see the stack trace of this error execute with --v=5 or higher
root@k8s-worker1:~# kubeadm join 172.31.30.85:6443 --token u9m3ka.7enokwokx7m5wk1m --discovery-token-ca-cert-has
h sha256:313004e1b8b297c63435816f7152dcb6ca23f94d946de96af23629624bc4438d
[preflight] Running pre-flight checks
[preflight] Reading configuration from the "kubeadm-config" ConfigMap in namespace "kube-system"...
[preflight] Use 'kubeadm init phase upload-config kubeadm --config your-config-file' to re-upload it.
W1106 10:31:28.673530    2240 utils.go:69] The recommended value for "bindAddress" in "KubeProxyConfiguration" i
s: ::; the provided value is: 0.0.0.0
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/instance-config.yaml"
[patches] Applied patch of type "application/strategic-merge-patch+json" to target "kubeletconfiguration"
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-check] Waiting for a healthy kubelet at http://127.0.0.1:10248/healthz. This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 503.252742ms
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap

This node has joined the cluster:
* Certificate signing request was sent to apiserver and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.

root@k8s-worker1:~# ^C
root@k8s-worker1:~#
```

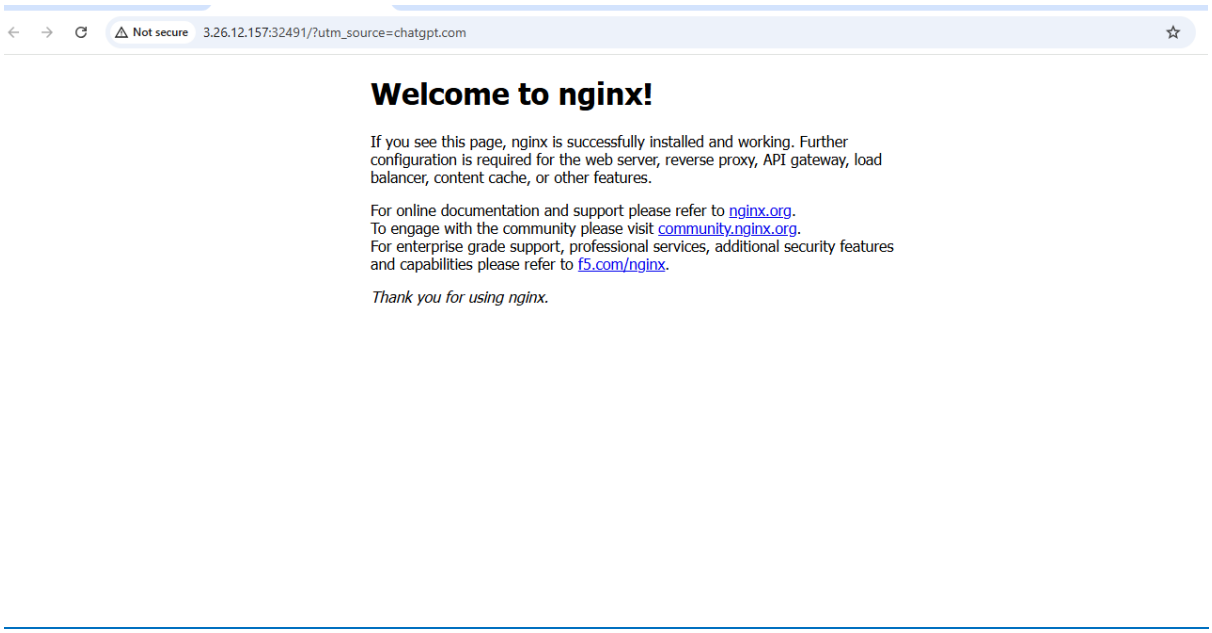
```
ags.env"
[kubelet-start] Starting the kubelet
[kubelet-check] Waiting for a healthy kubelet at http://127.0.0.1:10248/healthz. This can take u
p to 4m0s
[kubelet-check] The kubelet is healthy after 502.160544ms
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap

This node has joined the cluster:
* Certificate signing request was sent to apiserver and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.
```

Exposing Nginx Application Using NodePort

1. Created a NodePort service for the Nginx deployment.
2. Kubernetes assigned NodePort `32491`.
3. Configured AWS Security Group to allow inbound traffic.
4. Accessed the application using the worker node's public IP and NodePort.




```
root@k8s-master:~# kubectl create deployment nginx --image=nginx
deployment.apps/nginx created
root@k8s-master:~# kubectl get deployments
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
nginx   1/1     1            1           8s
root@k8s-master:~# kubectl get pods -o wide
NAME                     READY   STATUS    RESTARTS   AGE   IP           NODE          NOMINATED NODE
 READINESS GATES
nginx-66686b6766-ffc2q   1/1     Running   0          17s   10.244.2.3   k8s-worker1   <none>
 <none>
root@k8s-master:~# kubectl expose deployment nginx --port=80 --type=NodePort
service/nginx exposed
root@k8s-master:~# kubectl get svc
NAME         TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
kubernetes   ClusterIP   10.96.0.1        <none>        443/TCP        66m
nginx        NodePort    10.101.139.199   <none>        80:32491/TCP   14s
root@k8s-master:~# 3kubectl get svc nginx
3kubectl: command not found
root@k8s-master:~# kubectl get svc nginx
NAME    TYPE       CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
nginx   NodePort   10.101.139.199   <none>        80:32491/TCP   64s
root@k8s-master:~# kubectl get svc nginx
NAME    TYPE       CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
nginx   NodePort   10.101.139.199   <none>        80:32491/TCP   4m54s
root@k8s-master:~#
```

Kubernetes Deployment Scaling and Replica Management

1. Scaled the Nginx deployment from one replica to three replicas.
2. Used the `kubectl scale` command to increase pod replicas.
3. Verified the deployment status.
4. Checked pod distribution across worker nodes.
5. Confirmed all three replicas are running successfully.

